Practical: 7

Date: 8/10/2026

Addressing overfitting and underfitting using validation curve

Procedure:

1. Import the required Python libraries such as Pandas, NumPy, Matplotlib, Scikit-learn, and the required modules for preprocessing, model building, train-test splitting, learning curve, and validation curve.
2. Load the Wisconsin Diagnostic Breast Cancer (WDBC) dataset using pandas.read_csv().
3. Separate the dataset into independent variables (X) and the dependent variable (y).
4. Convert the categorical target variable into numerical form using LabelEncoder.
5. Divide the data into training and testing sets using train_test_split() with 80% training data and 20% testing data.
6. Create a Pipeline containing StandardScaler () for standardization and Logistic Regression as the classification model.
7. Use the learning curve to calculate training and validation accuracy for different training sample sizes.
8. Calculate the mean and standard deviation of the training and validation scores.
9. Plot the training and validation accuracy against the number of training samples. The graph helps to identify whether the model is suffering from overfitting or underfitting.
10. Use the validation curve to study the effect of the Logistic Regression parameter C on model performance.
11. Specify different values of C and calculate the training and validation accuracy using 10-fold cross-validation.
12. Plot the training and validation accuracy against the values of C on a logarithmic scale.
13. Compare the training and validation curves to identify the suitable value of C and determine whether the model is overfitting or underfitting.
14. Display both graphs using plt.show () and interpret the results.

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

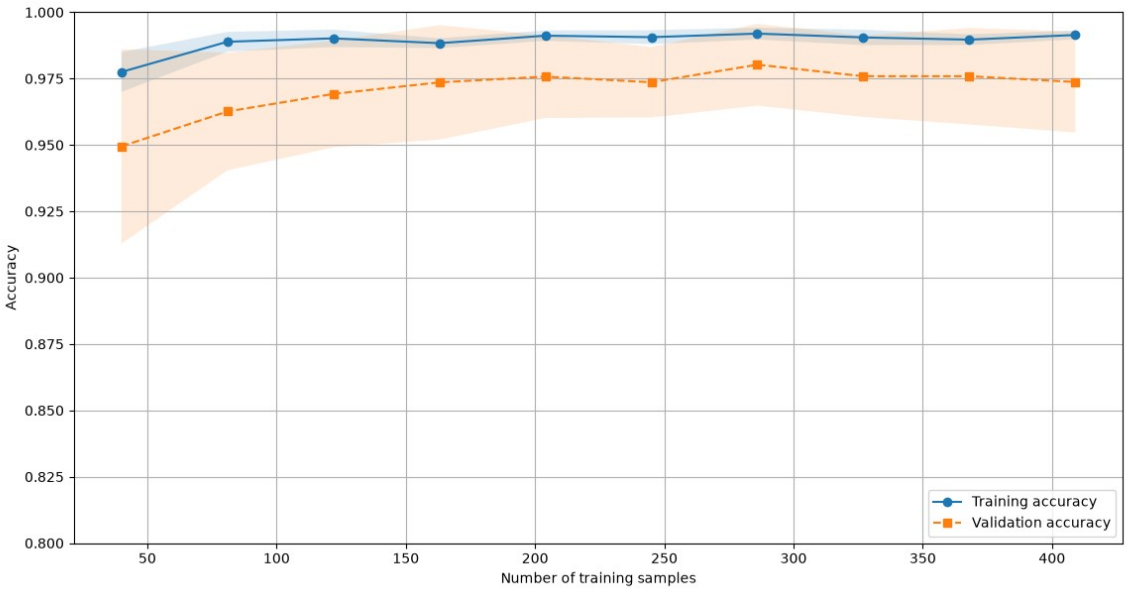
```
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, learning_curve,
validation_curve
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

df = pd.read_csv(r"C:/Users/Lenovo/Downloads/THEERTHA SDM College(Autonomous),
Ujire - wdbc (4).data", header=None)

X = df.iloc[:, 2:].values
y = df.iloc[:, 1].values
le = LabelEncoder()
y = le.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=1
)
pipe_lr = Pipeline([('scl', StandardScaler()),
    ('clf', LogisticRegression(C=1.0, random_state=0, max_iter=1000))])
train_sizes, train_scores, test_scores = learning_curve(
    estimator=pipe_lr,X=X_train,y=y_train,train_sizes=np.linspace(0.1, 1.0, 10),
    cv=10, n_jobs=1)
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.figure()
plt.plot(train_sizes, train_mean, marker='o', label='Training accuracy')
plt.fill_between(train_sizes,train_mean + train_std,train_mean - train_std,
    alpha=0.15)
plt.plot(train_sizes,test_mean, marker='s',linestyle='--',label='Validation
accuracy')
plt.fill_between(train_sizes,test_mean + test_std,test_mean - test_std,
    alpha=0.15)

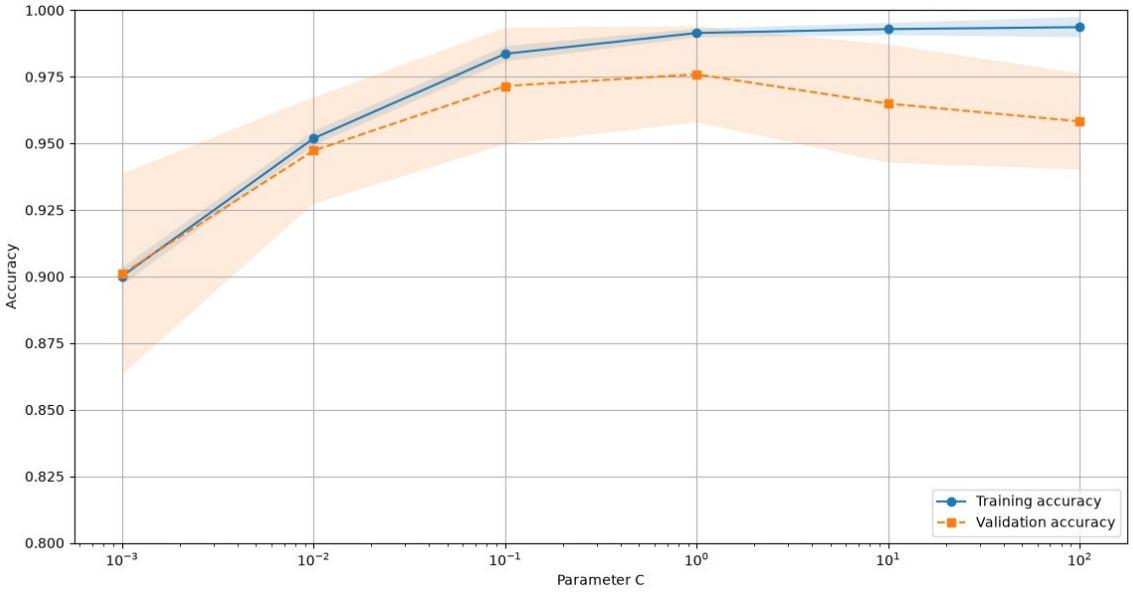
plt.grid()
plt.xlabel('Number of training samples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.0])
plt.show()
```




```
param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
train_scores, test_scores = validation_curve(
    estimator=pipe_lr,X=X_train,y=y_train,param_name='clf__C',
    param_range=param_range,cv=10)

train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.figure()
plt.plot(param_range,train_mean,marker='o',label='Training accuracy')
plt.fill_between(param_range,train_mean + train_std,train_mean - train_std,
    alpha=0.15)
plt.plot(param_range,test_mean,marker='s',linestyle='--',label='Validation
accuracy')
plt.fill_between(param_range, test_mean + test_std,test_mean - test_std,
    alpha=0.15)
plt.grid()
plt.xscale('log')
plt.xlabel('Parameter C')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.0])
plt.show()
```


1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
Accuracy
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
10^{2}
Parameter C
Training accuracy
Validation accuracy